

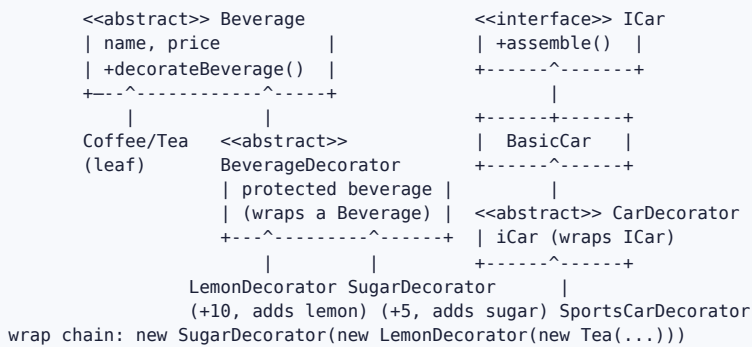
Decorator Design Pattern

Intent

Attach extra behavior to an object **dynamically, by wrapping it**, while keeping the same type — so a decorated object can be used anywhere the plain object is expected, and wrappers can stack (`new SugarDecorator(new LemonDecorator(new Tea(...)))`).

This module contains **two independent demonstrations**: a Beverage hierarchy (state-changing decorators that adjust name/price) and an ICar hierarchy (behavior-adding decorators around `assemble()`).

UML class diagram



The code, line by line

component/Beverage.java — the component (beverage side)

```
@Data
public abstract class Beverage {
    protected String name;
    protected int price;

    public Beverage() {
    }

    public Beverage(String name) {
        this.name = name;
    }

    public abstract void decorateBeverage();
}
```

- `@Data` (Lombok) — generates `getName()/setName()/getPrice()/setPrice()` used by the decorators; without it the decorator's `setName(...)/setPrice(...)` calls would not compile.
- `protected String name; protected int price;` — the state every beverage (and every decorator, since decorators **extend** this class) carries; `protected` so subclasses touch it directly.
- `public Beverage() {}` — no-arg constructor kept for subclass/framework use (its body is an empty IDE stub).
- `public Beverage(String name)` — stores the drink's display name.
- `public abstract void decorateBeverage();` — the operation the pattern decorates; leaves print their cost, decorators extend what happens.

component/concretComponent/Coffee.java and Tea.java — concrete components (leaves)

```
public class Coffee extends Beverage {
    public Coffee(String name) {
        super(name);
        setPrice(50);
    }

    @Override
    public void decorateBeverage() {
        System.out.println("The cost of " + name + ":" + price);
    }
}
```

- `super(name);` — passes the name up to `Beverage`.
- `setPrice(50);` — Lombok-generated setter; a plain Coffee costs 50 (Tea sets 20).
- `decorateBeverage()` — the leaf behavior: print name and current price. This is the call decorators will wrap.

decorator/BeverageDecorator.java — the abstract decorator (beverage side)

```
public abstract class BeverageDecorator extends Beverage {
    protected Beverage beverage;

    public BeverageDecorator(Beverage _beverage) {
        this.beverage = _beverage;
        setName(beverage.getName() + ":" + getDecoratedName());
        setPrice(beverage.getPrice() + getIncrementPrice());
    }

    @Override
    public void decorateBeverage() {
        System.out.println("Cost of " + name + ":" + price);
    }

    public abstract int getIncrementPrice();
    public abstract String getDecoratedName();
}
```

- `extends Beverage` — the decorator **is a** Beverage: this is what lets a wrapped drink be used wherever a drink is expected, and lets wrappers nest.
- `protected Beverage beverage;` — the decorator **has a** Beverage too: the wrapped delegate. IS-A + HAS-A the same type is the Decorator signature.
- Constructor line 1 `this.beverage = _beverage;` — stores the wrapped object.
- Constructor line 2 `setName(beverage.getName() + ":" + getDecoratedName());` — composes the display name from the wrapped drink's name plus this decorator's suffix. Note it calls the **abstract** `getDecoratedName()` — each concrete decorator supplies its own label (this also means decoration happens eagerly, at construction time).
- Constructor line 3 `setPrice(beverage.getPrice() + getIncrementPrice());` — adds this decorator's surcharge onto the wrapped price; stacking decorators accumulates price step by step.
- `decorateBeverage()` — default decorated print; concrete decorators override it and call `super` first.

decorator/concretDecorator/LemonDecorator.java and SugarDecorator.java

```
public class LemonDecorator extends BeverageDecorator {
    public LemonDecorator(Beverage _beverage) { super(_beverage); }

    @Override
    public void decorateBeverage() {
        super.decorateBeverage();
        decorateLemon();
    }
    public void decorateLemon() {
        System.out.println("Added Lemon to:" + beverage.getName());
    }
    @Override public int getIncrementPrice() { return 10; }
    @Override public String getDecoratedName() { return "lemon"; }
}
```

- `super(_beverage);` — triggers the base-decorator constructor, which composes name and price immediately.
- `decorateBeverage()` — the decorated operation: **first** `super.decorateBeverage()` prints the combined cost, **then** `decorateLemon()` adds this wrapper's own behavior. Behavior-before/after-delegate is the essence of Decorator.
- `getIncrementPrice()=10 / getDecoratedName()="lemon"` — this decorator's contribution (Sugar returns 5 / "Sugar").

component/ICar.java , concretComponent/BasicCar.java — the component (car side)

```
public interface ICar { public void assemble(); }

public class BasicCar implements ICar {
    @Override public void assemble() { System.out.println("Basic Car."); }
}
```

- The car side shows the same pattern with an **interface** component instead of an abstract class: `ICar` declares the operation, `BasicCar` is the undecorated leaf.

decorator/CarDecorator.java and concretDecorator/SportsCarDecorator.java

```

public abstract class CarDecorator implements ICar {
    public ICar iCar;
    public CarDecorator(ICar _iCar) { this.iCar = _iCar; }
    @Override public void assemble() { this.iCar.assemble(); }
}

public class SportsCarDecorator extends CarDecorator {
    public SportsCarDecorator(ICar _iCar) { super(_iCar); }
    @Override
    public void assemble() {
        super.assemble();
        System.out.println("Adding features of sports car.");
    }
}

```

- `CarDecorator` implements `ICar` + field `ICar iCar` — same IS-A + HAS-A signature.
- Base `assemble()` just **forwards** to the wrapped car — a pure pass-through decorator.
- `SportsCarDecorator.assemble()` — delegates first (`super.assemble()` → wrapped car assembles), then adds sports features: behavior added around the delegate without modifying `BasicCar`.

DesignPatternsApplication.java — the demo

```

public static void main(String[] args) {
    SpringApplication.run(DesignPatternsApplication.class, args);

    Coffee coffee = new Coffee("Cappuccino");
    coffee.decorateBeverage();

    Tea tea = new Tea("Black Tea");
    tea.decorateBeverage();
}

```

- The active demo only exercises the **plain leaves** (prints the cost of an undecorated Coffee and Tea).
- The commented-out lines are the actual decorator demos — e.g. `new SugarDecorator(new LemonDecorator(new Tea("Assam Tea")))` would print the stacked cost (20+10+5=35, name `Assam Tea:lemon:Sugar`) then "Added Lemon..." / "Added Sugar..." — and `new SportsCarDecorator(new BasicCar()).assemble()` for the car side. Uncomment them to see the wrapping in action.
- `SpringApplication.run(...)` boots Spring but plays no role in the pattern.

Why the design decisions

- **Why extend/implement the component?** So the decorated object is substitutable for the plain one — client code that accepts a `Beverage / ICar` works with any depth of wrapping.
- **Why hold the component too?** The wrapper must delegate to *something*; holding the wrapped instance lets each layer add its bit and pass the call on.
- **Why abstract decorator classes?** `BeverageDecorator / CarDecorator` centralize the wrapping plumbing (field, constructor, default delegation) so concrete decorators only state *what they add* (+10 / lemon , sports features).
- **Beverage vs Car nuance:** the beverage decorator does its work **in the constructor** (name/price composed eagerly); the car decorator does its work **in the method** (delegation at call time). The car form is the classic GoF shape; the beverage form shows decoration of *state* as well as behavior.
- **Honest notes:** the `// TODO Auto-generated ...` comments are leftover IDE stubs; the real decorator demo lines in `main` are commented out; `BasicCar / SportsCarDecorator` are therefore never exercised by the current run.

Relationship to the production code

This mirrors **dbaas-common-libs : MultiLevelCache** — a Spring `Cache` that wraps two other `Cache` delegates (Caffeine L0 + Redis L1) and layers tiered lookup/statistics on top, exactly the IS-A + HAS-A shape of `CarDecorator`. See `Design-Pattern-RealUseCase/RealUseCase_Structural/RealUseCase_Decorator_Design_Pattern` for the distilled version.